Your First ZipperGen Workflow

Draft a reply, ask a person, then send

The ZipperGen Authors

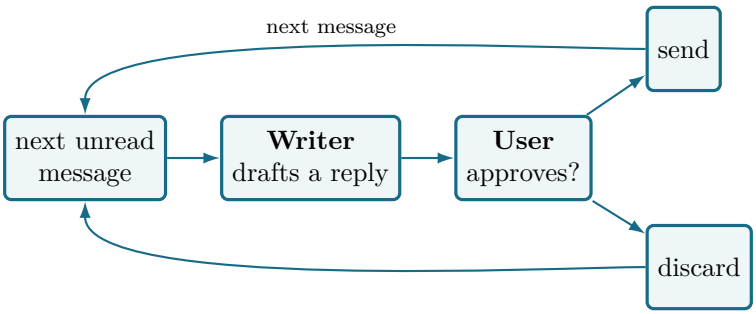
Tutorial edition: 8 August 2026

Abstract

You will build a small application that watches a mailbox, asks a language model to draft a reply to each message, and asks *you* to approve it before anything is sent. It does not stop on its own. It is a service, and the last section leaves it running on a server. It is short enough to read in one sitting, and it already contains everything that makes ZipperGen worth using: two participants, a model call, an explicit human decision, a branch that only the decider takes, and a loop that keeps the whole thing alive.

There is no separate ZipperGen development environment to learn. You work in an ordinary directory with an ordinary command line, and, if you like, a coding agent such as Claude Code or Codex. This tutorial shows both, and is honest about which parts need a person.

1 What you are building



It then waits for the next message, and does it again. Nothing ends the loop, which is what makes this a service rather than a script.

Concretely, you want this to happen:

Incoming message

Could we move our meeting to Thursday afternoon?

Proposed reply

Thursday afternoon works for me. How about 3pm?

`[ Send ] [ Reject ]`

Two participants exchange messages, one of them is a person, and what happens next depends on what that person says. That is a coordination problem, and it is the kind of thing ZipperGen exists to make readable.

What you need

- Python 3.11 or newer and ZipperGen installed.
- Nothing else for the first two thirds. The model is mocked and the approval happens in your terminal.
- For the last section, a Telegram bot token, if you want the approval to arrive on your phone.

2 Create the project

```
mkdir email-approval && cd email-approval
zippergen init
```

```
ZipperGen project: email-approval
  zippergen.toml     created
  specification.md   created
  AGENTS.md          created
  CLAUDE.md           created
```

`zg` is short for `zippergen`

Both commands are installed and do the same thing. The rest of this tutorial uses `zg`, because the commands you type often are the ones worth shortening.

That is the whole of project creation. It writes four files and stops: it asks you nothing, and it configures nothing.

- `zippergen.toml` is the project's visible configuration. It is committed, and it never contains a credential.
- `specification.md` says what the workflow is meant to do, in prose.
- `AGENTS.md` tells a coding agent to run `zippergen skill` before touching workflow code.
- `CLAUDE.md` points Claude Code at the same shared instructions.

3 Describe what you want

You can write the workflow yourself, and the next section shows the code. But the intended path is to say what you want in ordinary language. Open a coding agent in this directory:

```
claude
```

and ask for it:

```
> Build a ZipperGen workflow that watches a mailbox directory, asks an LLM
  to draft a short reply to each new message, and asks me to approve it
  before it is sent. It should keep running and wait for the next message.

  Use plain .txt files directly inside mailbox/. A file containing only
  "Can we meet on Thursday" is a complete message. Do not require From or
  Subject headers, and do not add an inbox/ subdirectory. Use mailbox/ as the
  local default rather than a required workflow input. The exact command
```

```
  "zg run --llm mock" must reach the approval without asking a setup question.
  Before reporting success, test that exact file, layout, and command.
```

The agent reads `AGENTS.md`, which points it at `zg skill`. Those are ZipperGen's own instructions for how to write a workflow, what to check, and what never to touch. It writes the files, then checks its work with the same commands you would use:

```
I created specification.md and workflow.py.

  $ zg validate
  Workflow email_approval: valid

Two participants, User and Writer, and one decision that the User owns:
whether to send the draft. Ask me to change anything.
```

That is the shape of the whole thing. You describe what you want. The agent edits ordinary files and uses ZipperGen's actions to check them. Nothing happened that you could not have done by hand, and everything it did is visible in the directory.

Where the skill comes from

`zg skill` prints instructions that ship inside the package, so they match the version you have installed. They are ordinary prose in the repository, not generated text. You can read them, and you can disagree with them. An already open Claude or Codex conversation does not refresh its context when ZipperGen is upgraded; start a new session, or explicitly ask it to run `zg skill` again.

---

4 What comes out

Visible, readable Python. This is the whole coordination:

```python
@workflow
def email_approval() -> int:
    User: message = next_unread_message()
    while message @ User:
        User(message) >> Writer(message)
        Writer: draft = draft_reply(message)
        Writer(draft) >> User(draft)
        User: approved = approve_reply(draft)
        if approved @ User:
            User: handled = send_reply(draft, handled)
        else:
            User: handled = discard(handled)
        User: message = next_unread_message()
    return handled @ User
```

Read it top to bottom and you have the entire message order. Five things are worth noticing.

`User(message) >> Writer(message)` is a message. The left side sends, the right side receives, and the names in brackets say which variables carry the value.

`Writer: draft = draft_reply(message)` is an action performed by one participant. `draft_reply` is an `@llm` action, which is a model call with a prompt and a declared output.

`User: approved = approve_reply(draft)` is a `@human` action. It stops and waits for a person.

`if approved @ User` is a decision, and the `@ User` says who makes it. That annotation is the important one, and the next section shows why.

`while message @ User` is a loop, owned the same way. The `User` decides each round whether there is another message to handle. `next_unread_message` waits until there is one, so in practice that decision is always yes and the workflow never finishes. That is the point.

One action hides the awkward part

Mailboxes are fiddly: what to do when nothing is unread, how to avoid handling the same message twice, whether to poll or block. All of it sits behind `next_unread_message`, whose contract is one sentence: *return the next message, waiting if there is none.* Here it takes the oldest `.txt` file from `mailbox/` and renames it `.done`, so no message is handled twice even after a crash. The coordination above does not change when you point it at a real inbox.

5 Check it yourself

Everything the agent ran, you can run:

```
zg validate
```

```
Workflow email_approval: valid
OK   lifelines: 2 unique lifeline(s): Writer, User
OK   projection Writer: WhileRecvStmt
OK   projection User: SeqStmt
OK   deployment declaration: 0 field(s), 0 package(s), 0 setup step(s)
OK   workflow inputs: none, the run starts without setup questions
OK   canonical rendering: global and local code views rendered successfully
```

`validate` loads the module, projects the protocol for every participant, and renders it back. If it passes, the workflow is well formed and the deadlock-freedom guarantee applies to it.

There is no privileged mode here. The agent and you have the same commands. Credentials are the exception, and you supply those yourself. The rest of this tutorial mixes both: asking the agent when a question is easier to say than to type, and running commands directly when they are short.

6 The moment that matters

The workflow has one decision and the `User` makes it. Rather than reading the code again, ask:

```
> What does the Writer actually run?
```

```
  $ zg show --agent Writer

  @role('Writer')
  def email_approval__Writer() -> None:
      while recv_decision('User'):
          message = recv('User')
          draft = draft_reply(message)
          send('User', draft)

The Writer is told each round whether to keep going, then drafts one
reply. It is never told whether the reply was sent: the User owns the
approval decision, and the Writer takes no part in it.
```

Run it yourself and you get the same four lines. Now compare the `User`, who owns both the loop and the decision:

```
zg show --agent User
```

```
@role('User')
def email_approval__User() -> int:
    message = next_unread_message()
    while message:
        send_decision('Writer', True)
        send('Writer', message)
        draft = recv('Writer')
        approved = approve_reply(draft)
        if approved:
            handled = send_reply(draft, handled)
        else:
            handled = discard(handled)
        message = next_unread_message()
    else:
        send_decision('Writer', False)
    return handled
```

The Writer has no approval branch

Nobody wrote either of those lines. Put the two programs side by side and the `User` does something for each of its two control structures, while the `Writer` does something for only one of them.

The loop, the `Writer` must know about: it has work inside it, so the `User` tells it every round whether to continue, and tells it once more, at the end, not to. That is the `send_decision` in the `else`.

The approval, the `Writer` must *not* know about: it does nothing in either branch, so the decision is erased from its program entirely. It cannot wait on it, disagree with it, or deadlock against it.

In a hand-written multi-agent system this is exactly where the bugs live: one agent waits for a message the other decided not to send. Here the question does not arise, because the projection worked out who needed telling and who did not.

This is not a convenience. It is the construction the correctness proof is about: the projected local programs produce exactly the behaviours of the global protocol, and deadlock freedom follows for well-formed workflows.

7 Run it

First put something in the mailbox. A message is just a text file:

```
mkdir -p mailbox
echo "Can we meet on Thursday" > mailbox/01.txt
```

The files go directly in `mailbox/`. There is no `mailbox/inbox/` directory. A plain one-line file is the message body, so headers such as `From` and `Subject` are optional.

Validation also makes unexpected setup inputs visible. For this workflow it must report:

```
OK   workflow inputs: none, the run starts without setup questions
```

No model key needed yet. `--llm mock` answers model calls with a placeholder:

```
zg run --llm mock
```

```
Proposed reply:

[draft_reply:draft]

Send this reply? [y/n]: y
    sent: [draft_reply:draft]
```

Then nothing happens, and that is correct: it is waiting for the next message. Drop another file into `mailbox/` from a second terminal and it wakes up and handles that one too. Answering `n` takes the other branch.

`mailbox/01.txt` is now `mailbox/01.done`. That rename is what stops a message being answered twice, including after a crash.

Stop it with `Ctrl-C`. When you want a run that ends by itself, in a test or just to see a result, give it a budget:

```
zg run --llm mock --option max_messages=2
```

```
{"result": 1}
```

Two messages handled, one of them approved.

> **Testing a specific branch**
>
> `mock` answers every action the same way, so a workflow driven by it takes one path and no other. When you want to drive a particular branch, such as the rejection, script the answers with `--llm scripted:replies.json`. The guide covers it. You do not need it yet.

8 Use a real model

Create a named configuration and assign the Writer to it:

```
zg model configure writer openai:gpt-4o-mini
zg model assign Writer writer
zg model
zg run
```

The model identity and assignment are written to `zippergen.toml`, so they travel with the project. If no key is available, the first command asks for `OPENAI_API_KEY` without echo and saves it privately on this computer. You can instead export the key before running the command. Several participants can share the same named configuration, and one exact action can be assigned with a target such as `Writer.draft_reply`. `zg run --llm mock` remains useful as a temporary override.

When working by hand, you can also run `zg model configure` and `zg model assign` with no values. ZipperGen then asks the questions and shows the available choices. It asks for the provider and model separately. Full commands keep the compact `PROVIDER:MODEL` form and remain better for scripts and coding agents.

9 Approval on your phone

So far the approval has appeared in whichever terminal is running the workflow. That is right while you are developing and wrong for something left running. Ask for the change:

```
> I want the approval to arrive on Telegram instead.
```

```
Done, as far as I can take it. The workflow itself does not change --- the
User's @human action is already the approval; what changes is where the
question is delivered, which is project configuration.

I have updated specification.md to say approvals go to Telegram, and
zg validate still passes.

One setup remains, and it belongs in your terminal. I should not ask you
for the chat id or bot token in this conversation. Run:

    zg connector configure approval-chat telegram
    zg connector assign User approval-chat

Tell me when that is done and I will check the configuration.
```

The agent can display the complete result with `zg config`, which does not contact providers. Use `zg check` when you want ZipperGen to verify real readiness; it contacts configured providers and may send a small model request.

That is the boundary, and it is worth being precise about it. The agent did everything it could and then stopped. The token is entered interactively in your terminal rather than passed as a command-line argument, so it does not appear in the agent's conversation, in your shell history, or in the repository.

That is a real boundary, and it is not an absolute one. Once saved, the token lives in a file under `ZIPPERGEN_HOME` readable by your user, so a coding agent running as you, with a shell, could read it afterwards. If you want a hard capability boundary rather than a convention, run the agent without access to your credential store: a separate account, or an environment that does not carry `ZIPPERGEN_HOME`.

Coding agent

```
> I want the approval to arrive
  on Telegram instead.

Updated the specification.
zg validate passes.

Telegram setup needs your terminal:

    zg connector configure
      approval-chat telegram
    zg connector assign User
      approval-chat
```

Your terminal

```
$ zg connector configure
    approval-chat telegram

Telegram chat id: 12345678
Telegram bot token (hidden):
Saved the token for this computer.
Saved configuration approval-chat.

$ zg connector assign User
    approval-chat
User will be asked through
  approval-chat.
```

Here `approval-chat` is a name you choose, not a ZipperGen keyword. The first command creates that named configuration using the `telegram` provider and says *which* chat. The second says *who* is asked there: the `User`, whose `@human` action is the approval.

And two things went to two different places, on purpose:

chat id	written to `zippergen.toml`, committed, shared
bot token	typed, never an argument, stays on this computer

A colleague who clones the project gets the chat id and supplies their own token. The token never reaches your shell history, the repository, or the agent.

Authorizing Google from another computer

The same split appears for Gmail and Sheets, with one extra step because a server usually has no browser. Run `zg connector authorize google --scopes ...` on your own machine. It opens a browser and prints an encoded result. Paste that into `zg connector accept google` on the server. The credential never appears in a command line.

10 Runs that survive interruption

There is one verb for running a workflow. `--durable` records the run as it goes, so an interrupted run continues rather than starting over:

```
zg run --durable --llm mock
```

```
Workflow email_approval: valid
Run email_approval-20260808-132719-612865000
Proposed reply:
...
```

Press Ctrl-C while it waits for your approval, then:

```
zg run inspect --agent Writer
zg run --resume
```

The model result and control position committed before the interruption are preserved, so this run resumes at the approval rather than drafting again. A crash during an external call can still repeat that call if its result had not yet committed.

To abandon the selected run, stop it with Ctrl-C and use `zg run reset`. Its record and SQLite state are permanently discarded and no run remains current; add `--archive` only when a recoverable private copy is wanted. This does not start another run; use `zg run --durable` when one is wanted. Starting a new durable run also discards an older selected development run.

While a durable run is active in one terminal, another terminal can keep its program position open and refresh it once per second:

```
zg run inspect --watch --agent Writer
```

Pressing Ctrl-C in this view closes only the view. It does not interrupt the run.

This matters more for a workflow that never ends than for one that finishes in a second. A foreground durable run continues with `zg run --resume`; a deployment is restarted by its service manager. In both cases committed state prevents it from starting the workflow over after an ordinary stop.

11 Deploy it

The ordinary deployment command prepares, checks, and starts the service. For this example it first asks for the mailbox directory; enter the project directory you used above, such as `mailbox`. ZipperGen records its absolute path so the service does not accidentally watch an empty directory inside its immutable source bundle:

```
zg deploy
```

Every model, assistant CLI, and connector is probed first, and a failure stops the deployment before it starts:

```
zg deploy logs
zg deploy status
zg deploy inspect --watch
```

Use `zg deploy --no-start` only when you deliberately want to prepare and review a stopped deployment before a later `zg deploy start`. It is not a required preliminary step.

A deployment you no longer want:

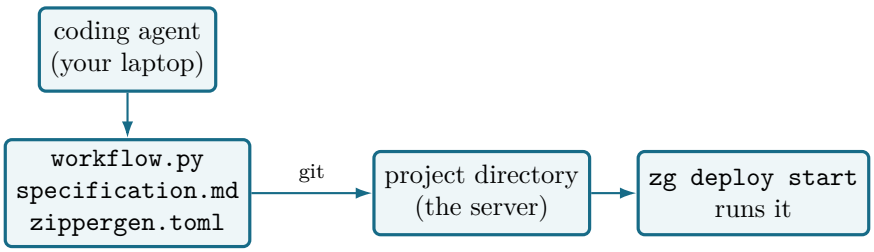
```
zg deploy remove
```

Its durable store is kept. That is the record of what actually ran, and it is the one thing no rebuild can recover. Everything else, the environment and service files and the bundled source, is written again by deploying again. `--purge` removes the store too, when you mean it.

12 Put it on a server

This section is optional. Everything above works on your laptop. Read on only when you want the workflow to keep running after you close it.

There is no ZipperGen deployment host and no server to sign up for. You copy the project to a machine, install ZipperGen there, and run the same commands you have just been running. That is the whole mechanism.



Notice what does *not* cross to the right. The coding agent stays on your laptop. It wrote the workflow, but it is not part of the running application, and the server never needs Claude, Codex, or an agent of any kind installed. The server needs three things: ZipperGen, the project files, and its own credentials.

On an ordinary Linux machine:

```
git clone https://github.com/me/email-approval.git && cd email-approval
python3 -m venv .venv && source .venv/bin/activate
pip install zippergen

zg connector configure approval-chat telegram
zg connector assign User approval-chat

zg deploy
```

The credentials did not arrive with the clone, because they were never in it. `configure` asks for the bot token, does not echo it, and stores it outside the project. That is why cloning gives the server the routing without giving it the secret.

The last command prepares, checks, and starts the deployment. It probes the model and connector first, so a token typed wrongly here fails now rather than in the middle of an approval. On Linux it

installs a small `systemd` user service that restarts the workflow if it fails. You never write that file, and `zg deploy status`, `zg deploy logs` and `zg deploy stop` manage it from here.

This is the point of the loop. The workflow you built does not finish, so leaving it on a server is not a contrivance to show off a `deploy` command. It is the only sensible place for it. Your laptop can close.

The mailbox here is a directory, which is enough to see the shape. Point `next_unread_message` at a real inbox and nothing above changes. `examples/call_intake.py` is that workflow, reading Gmail and writing to a spreadsheet.

13 What you have

`specification.md`	what it should do, in prose, maintained by you and your agent
`workflow.py`	the coordination, as visible Python
`zippergen.toml`	which model, which chat, committed, no secrets
`mailbox/`	where messages arrive, a real inbox on a real server

Three files and a directory, plus whatever your machine supplies: a model key in the environment, a bot token typed once.

The thing worth remembering is the `Writer`. It learned about the loop, because it has work to do inside it. It never learned about the approval, because it has none. You wrote one protocol, and ZipperGen worked out what each participant runs, and worked it out in a way that cannot deadlock. As the workflow grows, with more participants, more loops and parallel regions, that is what stops the coordination becoming the hard part.

14 Where to go next

- `examples/diagnosis.py` has two reviewers who argue until they agree. A loop, and a decision inside it.
- `examples/call_intake.py` reads a real mailbox and records to a real spreadsheet, runs as a service.
- *Development and deployment guide* is the longer reference.
- `zippergen skill` prints what your coding agent is following. Worth reading once yourself.